



## Nội dung trình bày:

1. Tổng quan MIPS 32bits
2. Thiết kế các khối chính trong MIPS
3. Thiết kế thanh ghi Pipeline
4. Thiết kế các khối xử lý hazard
5. Kiểm tra kết quả mô phỏng
6. Đánh giá phần cứng và hiệu năng



# 1. Tổng quan về MIPS 32bits:

Hệ thống chia việc thực thi lệnh thành 5 giai đoạn độc lập, cho phép xử lý song song nhiều lệnh cùng lúc:

**IF (Instruction Fetch):** Nạp lệnh từ bộ nhớ.

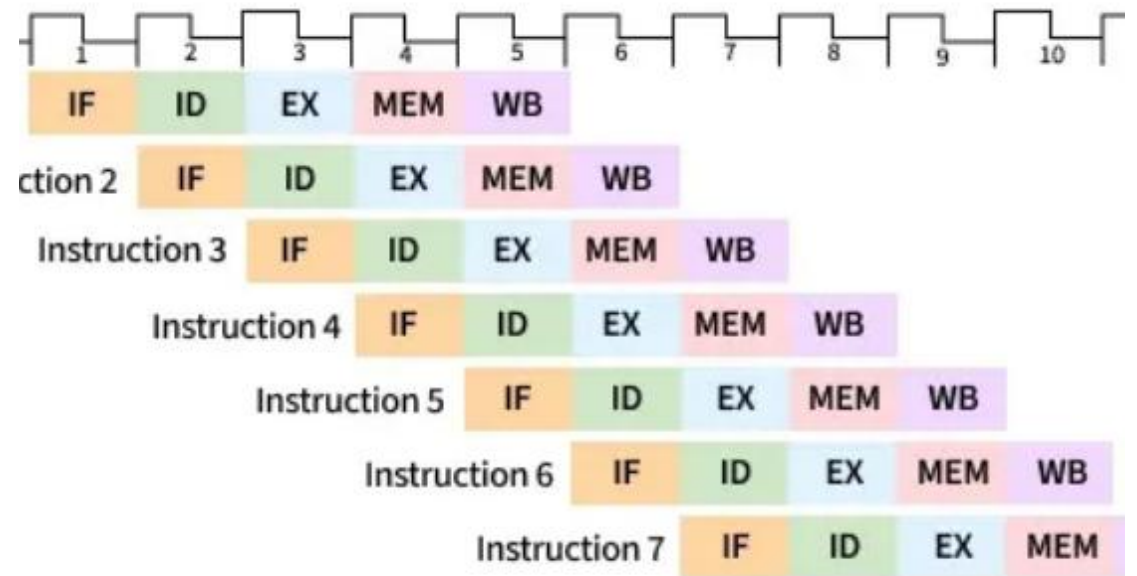
**ID (Instruction Decode):** Giải mã và đọc thanh ghi.

**EX (Execute):** Thực thi ALU và tính địa chỉ.

**MEM (Memory):** Truy xuất dữ liệu RAM.

**WB (Writeback):** Ghi kết quả về tập thanh ghi.

Instruction Execution In 5-Stage Pipeline





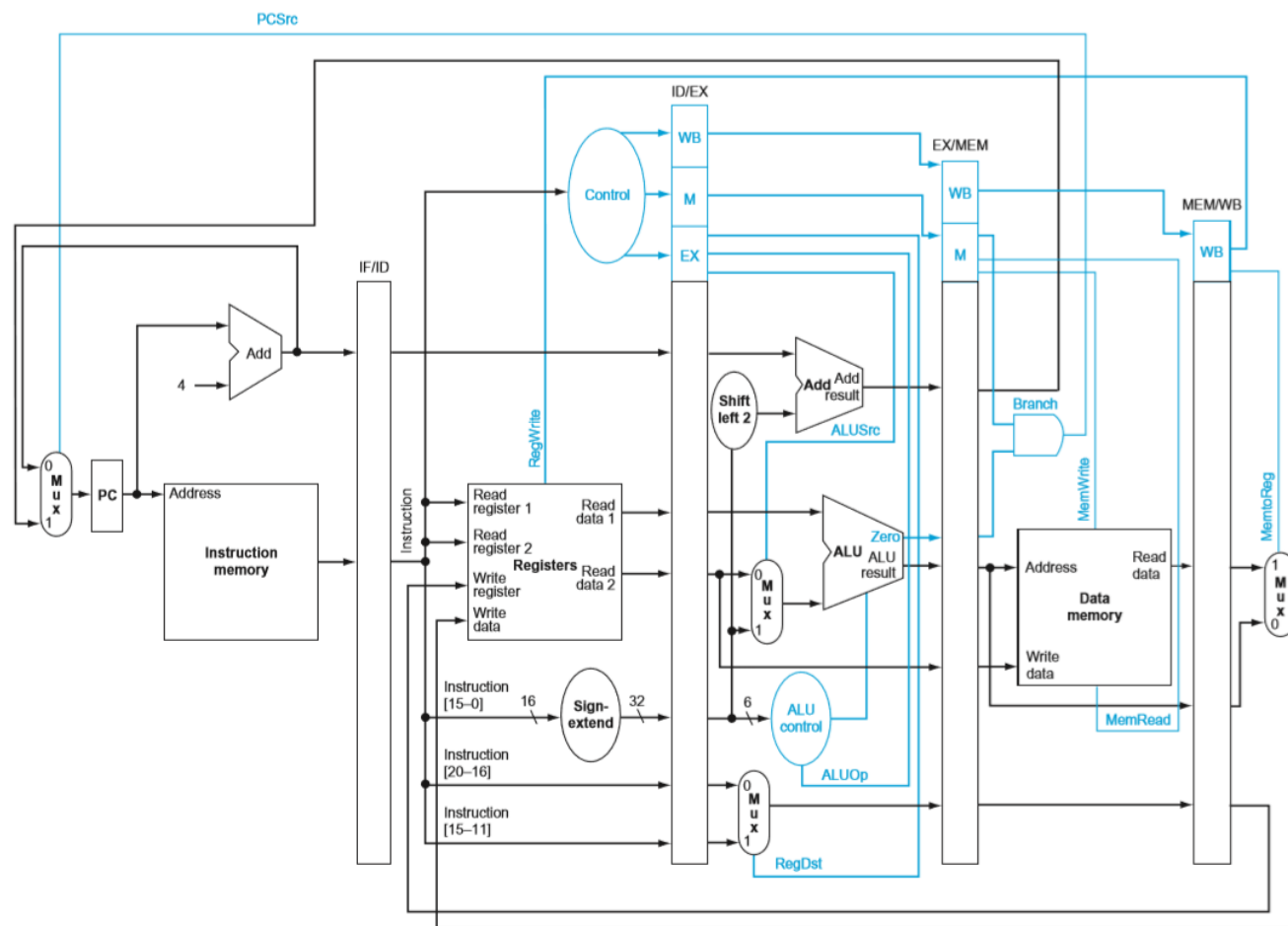
# 1. Tổng quan về MIPS 32bits:

| Tập lệnh      | Các lệnh                                   |
|---------------|--------------------------------------------|
| <b>R-Type</b> | Add, Sub, And, Or, Nor, Xor, Slt, Sll, Srl |
| <b>J-Type</b> | Jump                                       |
| <b>I-Type</b> | Addi, Andi, Ori, Lw, Sw, Beq, Bne          |



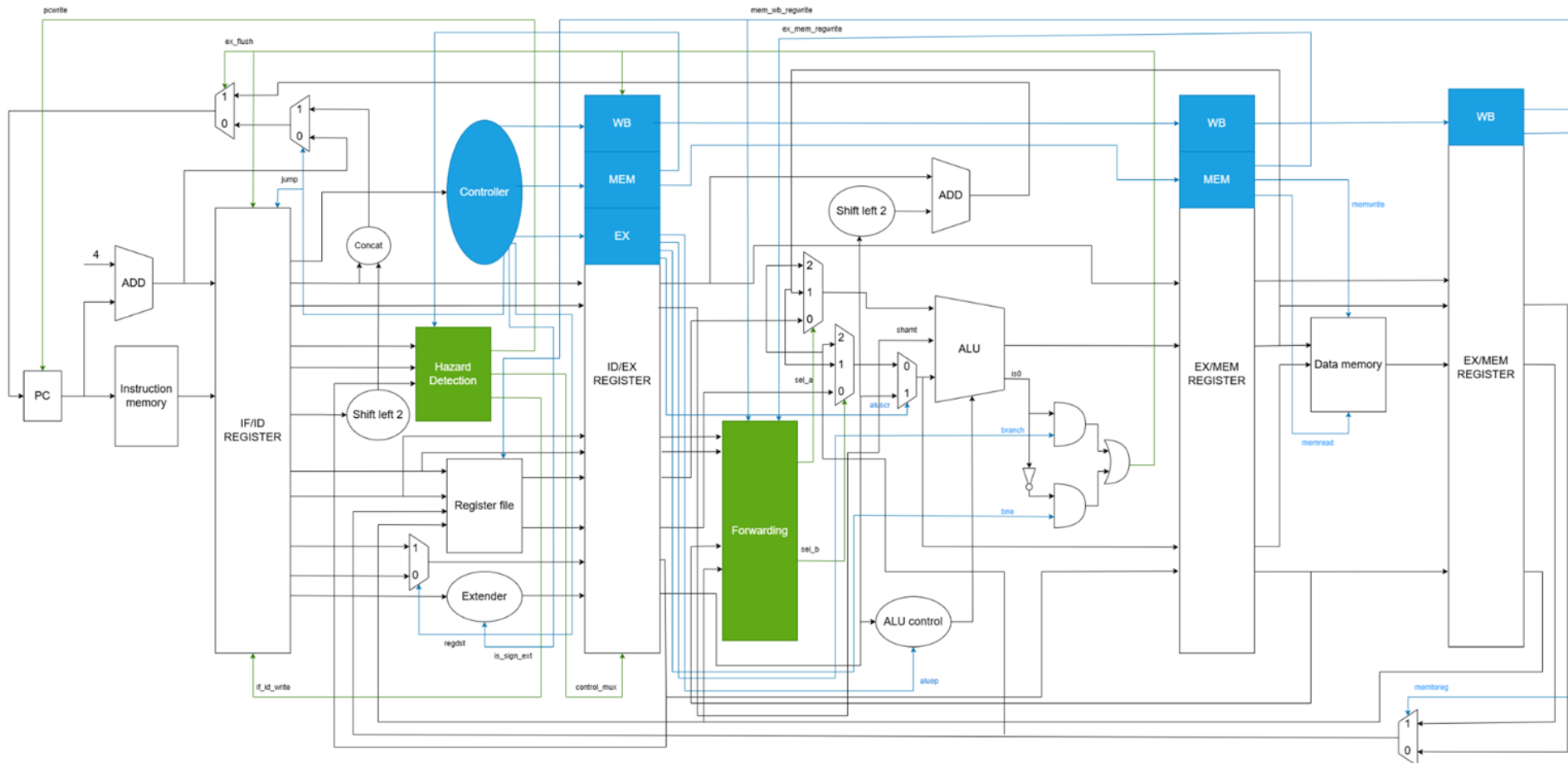
## 2. Thiết kế các khối chính trong MIPS:

Datapath tham khảo





## 2. Thiết kế các khối chính trong MIPS:





## 2. Thiết kế các khối chính trong MIPS:

### 2.1 Thiết kế PC:

```
always @(posedge clk or negedge
pcreset) begin
    if (!pcreset)
        pc <= 32'd0;
    else if (pcwrite)
        pc <= pc_next;
end
```

// Xác định địa chỉ nhảy ở Datapath

```
assign pc_4 = pc + 4;
assign pc_next = (ex_flush) ? branchpc :
(jump) ? jump_address : pc_4;
```

- Thanh ghi 32-bit cập nhật tại cạnh lên xung Clock. Hỗ trợ Reset bất đồng bộ để đưa PC về 0 khi khởi động.
- Tín hiệu pcwrite quyết định việc cập nhật. Cờ này bị kéo xuống 0 bởi để giữ nguyên PC khi gặp xung đột Load-Use.
- Dùng MUX để chọn giá trị cập nhật cho PC: Ưu tiên 1 cho rẽ nhánh (ex\_flush), Ưu tiên 2 cho lệnh nhảy (jump), mặc định là PC + 4.



## 2. Thiết kế các khối chính trong MIPS:

### 2.2 Thiết kế IMEM:

```
module IMEM (  
    input [31:0] readadr,  
    output [31:0] inst  
);  
  
    reg [31:0] mem [0:255];  
  
    initial begin  
        $readmemh("instruction.txt", mem);  
    end  
  
    assign inst = mem[readadr[31:2]];  
  
endmodule
```

- Khởi tạo 256 ô nhớ 32-bit. Nạp mã máy tự động bằng \$readmemh.
- Trích xuất lệnh tức thời theo PC, không cần chờ cạnh xung Clock.
- Dùng [31:2] (chia 4) để đổi địa chỉ Byte của PC sang chỉ số mảng (Index).



## 2. Thiết kế các khối chính trong MIPS:

### 2.3 Thiết kế Register File 32x32:

```
always @(posedge clk or negedge reset) begin
    if (!reset) begin
        for (i = 0; i < 32; i = i + 1) begin
            register[i] <= 32'd0;
        end
    end else if (regwrite && writereg != 5'd0) begin
        register[writereg] <= writedata;
    end
end
always @(*) begin
    readdata1 = (regwrite && (writereg == rs1) &&
        (writereg != 5'd0)) ? writedata : register[rs1];
    readdata2 = (regwrite && (writereg == rs2) &&
        (writereg != 5'd0)) ? writedata : register[rs2];
end
```

Giải quyết tình trạng lệnh ở tầng Write Back (WB) và tầng Decode (ID) truy xuất cùng một thanh ghi trong cùng 1 chu kỳ clock. Nếu địa chỉ đọc (rs1/rs2) trùng với địa chỉ đang ghi (writereg), mạch sẽ ưu tiên lấy thẳng giá trị mới writedata đẩy ra ngoài để tránh việc lệnh ID đọc phải dữ liệu cũ.



## 2. Thiết kế các khối chính trong MIPS:

### 2.4 Thiết kế Extender:

```
module Extender (  
    input [15:0] a,  
    input is_sign_ext,  
    output [31:0] b  
);  
    // is_sign_ext = 1: Sign Extend  
    // is_sign_ext = 0: Zero Extend  
    assign b = (is_sign_ext) ? {{16{a[15]}}, a}  
    : {16'd0, a};  
endmodule
```

- Mở rộng giá trị Immediate từ 16-bit lên 32-bit để đồng bộ kích thước dữ liệu đưa vào khối ALU.
- Tùy vào cờ `is_sign_ext`, mạch sẽ nhân bản bit dấu (cho lệnh số học/bộ nhớ như `addi`, `lw`) hoặc điền toàn số 0 (cho lệnh logic như `andi`, `ori`) vào 16 bit cao.



## 2. Thiết kế các khối chính trong MIPS:

### 2.5 Thiết kế ALU:

#### Thiết kế ALU Control:

```
always @(*) begin
    case (aluop)
        3'd0: aluctrl = funct;           // (R-Type)
        3'd1: aluctrl = 6'h20;           // Cộng có dấu
        3'd2: aluctrl = 6'h22;           // Trừ có dấu
        3'd3: aluctrl = 6'h24;           // Logic AND
        3'd4: aluctrl = 6'h25;           // Logic OR
        default: aluctrl = 6'h00;        // (jump)
    endcase
end
```

- Dựa vào mã aluop từ khối điều khiển chính để chốt phép toán cuối cùng cho ALU.
- **R-Type:** Đẩy thẳng trường funct ra ngõ aluctrl để ALU tự tính toán theo mã lệnh gốc.
- **I/J-Type:** Ép cứng giá trị aluctrl. VD: Ép thành lệnh Cộng (6'h20) để tính địa chỉ RAM, hoặc Trừ (6'h22) để kiểm tra nhảy beq.



## 2. Thiết kế các khối chính trong MIPS:

### 2.5 Thiết kế ALU:

```
always @(*) begin
  case (aluctrl)
    6'h00: result = b << shamt;
    6'h02: result = b >> shamt;
    6'h20: result = $signed(a) + $signed(b);
    6'h21: result = $unsigned(a) + $unsigned(b);
    6'h22: result = $signed(a) - $signed(b);
    6'h23: result = $unsigned(a) - $unsigned(b);
    6'h24: result = a & b;
    6'h25: result = a | b;
    6'h26: result = a ^ b;
    6'h27: result = ~(a | b);
    6'h2a: result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
    default: result = 32'd0;
  endcase
end
assign isz = (result == 32'd0) ? 1'b1 : 1'b0;
```

- Dựa vào aluctrl để thực thi các phép toán và dịch bit (Shift Left/Right) theo shamt.
- Tự động bật lên mức 1 khi kết quả tính toán bằng 0. Đây là cờ quyết định để bẻ lái nhảy ở các lệnh rẽ nhánh beq/bne.



## 2. Thiết kế các khối chính trong MIPS:

### 2.6 Thiết kế DMEM:

```
reg [31:0] dmem [0:255];  
  
assign readdata = (memread) ? dmem[address[31:2]] : 32'd0;  
  
always @(posedge clk) begin  
    if (memwrite) begin  
        dmem[address[31:2]] <= writedata;  
    end  
end
```

- Khởi tạo mảng dmem gồm 256 phần tử, mỗi phần tử rộng 32-bit (1 Word). Tổng dung lượng lưu trữ tương đương 1KB.
- Việc đọc dữ liệu diễn ra tức thời khi cờ memread bật. Còn việc ghi phải đợi cạnh lên của xung Clock để đảm bảo an toàn, tránh ghi rác vào RAM.



## 2. Thiết kế các khối chính trong MIPS:

### 2.7 Thiết kế Controller:

| <u>Lệnh</u> | <u>opcode</u> | <u>regdst</u> | <u>branch</u> | <u>bne</u> | <u>memread</u> | <u>memtoreg</u> | <u>aluop [2:0]</u> | <u>memwrite</u> | <u>alusrc</u> | <u>regwrite</u> | <u>jump</u> | <u>is sign_ext</u> |
|-------------|---------------|---------------|---------------|------------|----------------|-----------------|--------------------|-----------------|---------------|-----------------|-------------|--------------------|
| R-Type      | 000000        | 1             | 0             | 0          | 0              | 0               | 000                | 0               | 0             | 1               | 0           | 0                  |
| lw          | 100011        | 0             | 0             | 0          | 1              | 1               | 000                | 0               | 1             | 1               | 0           | 1                  |
| sw          | 101011        | 0             | 0             | 0          | 0              | 0               | 000                | 1               | 1             | 0               | 0           | 1                  |
| beq         | 000100        | 0             | 1             | 0          | 0              | 0               | 010                | 0               | 0             | 0               | 0           | 1                  |
| bne         | 000101        | 0             | 0             | 1          | 0              | 0               | 010                | 0               | 0             | 0               | 0           | 1                  |
| addi        | 001000        | 0             | 0             | 0          | 0              | 0               | 001                | 0               | 1             | 1               | 0           | 1                  |
| li          | 001001        | 0             | 0             | 0          | 0              | 0               | 001                | 0               | 1             | 1               | 0           | 1                  |
| andi        | 001100        | 0             | 0             | 0          | 0              | 0               | 011                | 0               | 1             | 1               | 0           | 0                  |
| ori         | 001101        | 0             | 0             | 0          | 0              | 0               | 100                | 0               | 1             | 1               | 0           | 0                  |
| jump        | 000010        | 0             | 0             | 0          | 0              | 0               | 000                | 0               | 0             | 0               | 1           | 0                  |



## 3. Thiết kế thanh ghi Pipeline

### 3.1 Thiết kế thanh ghi IF/ID

```
always @(posedge clk or negedge pcreset)
begin
    if(!pcreset) begin
        if_id_inst <= 32'd0;
        if_id_pc   <= 32'd0;
    end else if (ex_flush || jump) begin
        if_id_inst <= 32'd0;
        if_id_pc   <= 32'd0;
    end else if (if_id_write) begin
        if_id_inst <= inst_in;
        if_id_pc   <= pc_4_in;
    end
end
end
```

- Chốt lệnh (Instruction) và địa chỉ PC+4 giữa hai tầng IF và ID tại cạnh dương xung clock.
- Khi có lệnh Jump hoặc Flush (do rẽ nhánh), module sẽ xóa lệnh hiện tại thành NOP (giá trị 0) để tránh thực thi lệnh sai.
- Tín hiệu if\_id\_write cho phép giữ nguyên lệnh cũ khi gặp Hazard dữ liệu cần Stall.



## 3. Thiết kế thanh ghi Pipeline

### 3.2 Thiết kế thanh ghi ID/EX

```
always @(posedge clk or negedge pcreset) begin
    if (!pcreset) begin
        // 1. Reset cứng (Bất đồng bộ)
    end else if (ex_flush) begin
        // 2. Flush mềm (Đồng bộ)
    end else begin
        // 3. Chạy bình thường
        id_ex_branch      <= branch_in  & control_mux;
        id_ex_memread     <= memread_in & control_mux;
        id_ex_regwrite    <= regwrite_in & control_mux;

        id_ex_rs1         <= rs1_in;
        id_ex_data1       <= data1_in;
        id_ex_signex      <= signex_in;
        id_ex_shamt       <= shamt_in;
    end
end
```

- Là cầu nối trung chuyển dữ liệu và tín hiệu điều khiển từ tầng **Decode (ID)** sang tầng **Execute (EX)**.
- Sử dụng tín hiệu control\_mux để các tín hiệu điều khiển về 0 khi hệ thống cần **Stall** do Hazard dữ liệu.
- Tín hiệu ex\_flush sẽ xóa sạch nội dung thanh ghi khi phát hiện rẽ nhánh sai (Control Hazard), đảm bảo tính đúng đắn của luồng lệnh.



## 3. Thiết kế thanh ghi Pipeline

### 3.3 Thiết kế thanh ghi EX/MEM

```
always @(posedge clk or negedge
pcreset) begin
  if (!pcreset) begin
    // 1. Reset cứng (Bất đồng bộ)
  end else begin
    // 2. Chạy bình thường
    // Tín hiệu điều khiển
    ex_mem_memwrite <= memwrite_in;
    ex_mem_regwrite <= regwrite_in;
    // Dữ liệu
    ex_mem_result <= result_in;
    ex_mem_rd      <= rd_in;
```

- Chốt kết quả tính toán từ khối ALU (Result) và dữ liệu cần ghi vào bộ nhớ (WriteData) giữa hai tầng EX và MEM tại cạnh dương xung clock.
- Chuyển tiếp các tín hiệu điều khiển quan trọng cho tầng MEM (MemWrite, MemRead) và tầng WB (RegWrite, MemtoReg).
- Thanh ghi này **không có cơ chế Stall hay Flush**. Lý do là mọi xung đột (Data Hazard, Control Hazard) đều đã được xử lý ở các tầng trước.



## 3. Thiết kế thanh ghi Pipeline

### 3.4 Thiết kế thanh ghi MEM/WB

```
always @(posedge clk or negedge pcreset) begin
    if (!pcreset) begin
        //1. Reset cứng (Bất đồng bộ)
        mem_wb_alu_result <= 32'd0;
        mem_wb_mem_data    <= 32'd0;
        ...
    end else begin
        //2. Chạy bình thường
        mem_wb_alu_result <= alu_result_in;
        mem_wb_mem_data    <= mem_data_in;
        ...
    end
end
```

- Chốt dữ liệu đọc ra từ Data Memory (nếu là lệnh Load) và kết quả từ ALU chuẩn bị cho bước ghi cuối cùng tại cạnh dương xung clock.
- Chỉ giữ lại 2 tín hiệu điều khiển duy nhất cho Tầng Write Back: RegWrite (cho phép ghi) và MemtoReg (chọn nguồn dữ liệu từ ALU hay từ bộ nhớ để ghi).
- Tương tự như tầng EX/MEM, thanh ghi MEM/WB nằm ở cuối chu trình xử lý nên lệnh tới đây là hoàn toàn hợp lệ.



## 4. Thiết kế các khối xử lý hazard

### 4.1 Khối Forwarding

```
//sel = 0 Lấy dữ liệu gốc từ thanh ghi ID/EX
//sel = 1 Lấy kết quả ALU từ thanh ghi EX/MEM
//sel = 2 Lấy dữ liệu từ thanh ghi MEM/WB
always @(*) begin
    if(id_ex_rs1==ex_mem_rd && ex_mem_regwrite && ex_mem_rd!=0)
        sela = 1;
    else if (id_ex_rs1==mem_wb_rd && mem_wb_regwrite &&
mem_wb_rd!=0)
        sela = 2;
    else sela = 0;
    if(id_ex_rs2==ex_mem_rd && ex_mem_regwrite && ex_mem_rd!=0)
        selb = 1;
    else if (id_ex_rs2==mem_wb_rd && mem_wb_regwrite &&
mem_wb_rd!=0)
        selb = 2;
    else selb = 0;
end
```

end 26 May 2026

Copyrights 2026 UIT. All Rights Reserved.

- Khối tổ hợp này xuất ra tín hiệu điều khiển sela và selb để giải quyết xung đột dữ liệu bằng cách nối tắt dữ liệu thẳng vào ALU



## 4. Thiết kế các khối xử lý hazard

### 4.2 Khối Hazard Detection

```
always @(*) begin
    // Phát hiện xung đột Load-Use
    if (id_ex_memread && (id_ex_rd != 5'd0) &&
        ((id_ex_rd == if_id_rs1) || (id_ex_rd == if_id_rs2)))
    begin
        // Tạo bong bóng (Stall)
        pcwrite      = 0;
        if_id_write = 0;
        control_mux = 0;
    end else begin
        // Hoạt động bình thường
        pcwrite      = 1;
        if_id_write = 1;
        control_mux = 1;
    end
end
end
```

- Phát hiện và xử lý riêng biệt lỗi **Load-Use Data Hazard** (khi lệnh đang giải mã cần dùng ngay kết quả của lệnh lw đi sát trước nó)
- Khi phát hiện lỗi, khối sẽ ép pcwrite = 0 và if\_id\_write = 0 để đóng băng PC và thanh ghi IF/ID. Đồng thời kéo control\_mux = 0 để bơm một lệnh NOP xuống tầng EX

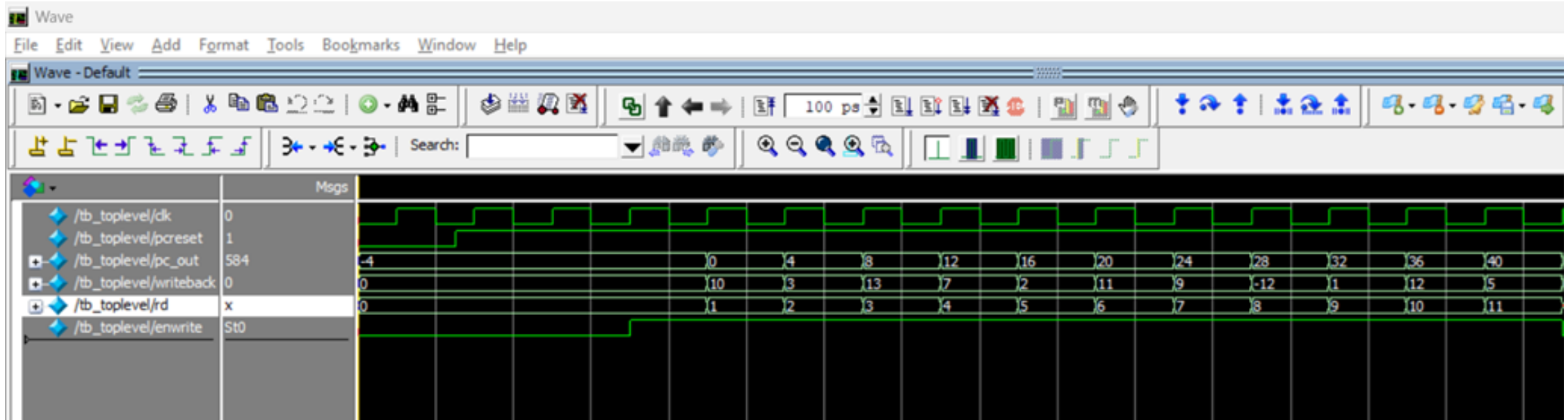


## 5. Kiểm tra kết quả mô phỏng

### 5.1 Test các lệnh R

```

testcase1.asm
1  # Nạp dữ liệu
2  addi $1, $0, 10      # $1 = 10
3  addi $2, $0, 3       # $2 = 3
4  # Test các lệnh R-Type
5  add  $3, $1, $2      # EXPECTED: $3 = 13
6  sub  $4, $1, $2      # EXPECTED: $4 = 7
7  and  $5, $1, $2      # EXPECTED: $5 = 2
8  or   $6, $1, $2      # EXPECTED: $6 = 11
9  xor  $7, $1, $2      # EXPECTED: $7 = 9
10 nor  $8, $1, $2      # EXPECTED: $8 = -12
11 slt  $9, $2, $1      # EXPECTED: $9 = 1
12 sll  $10, $2, 2      # EXPECTED: $10 = 12
13 srl  $11, $1, 1      # EXPECTED: $11 = 5
  
```

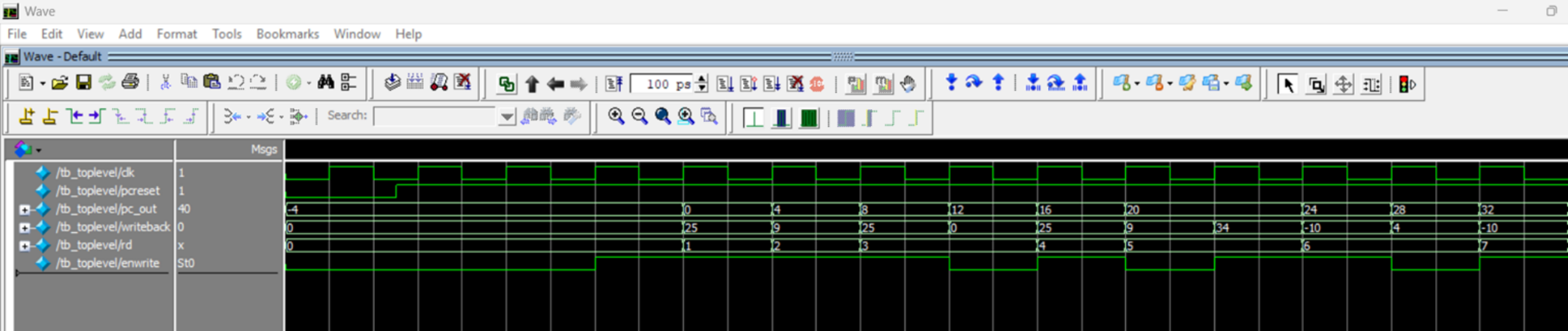




## 5. Kiểm tra kết quả mô phỏng

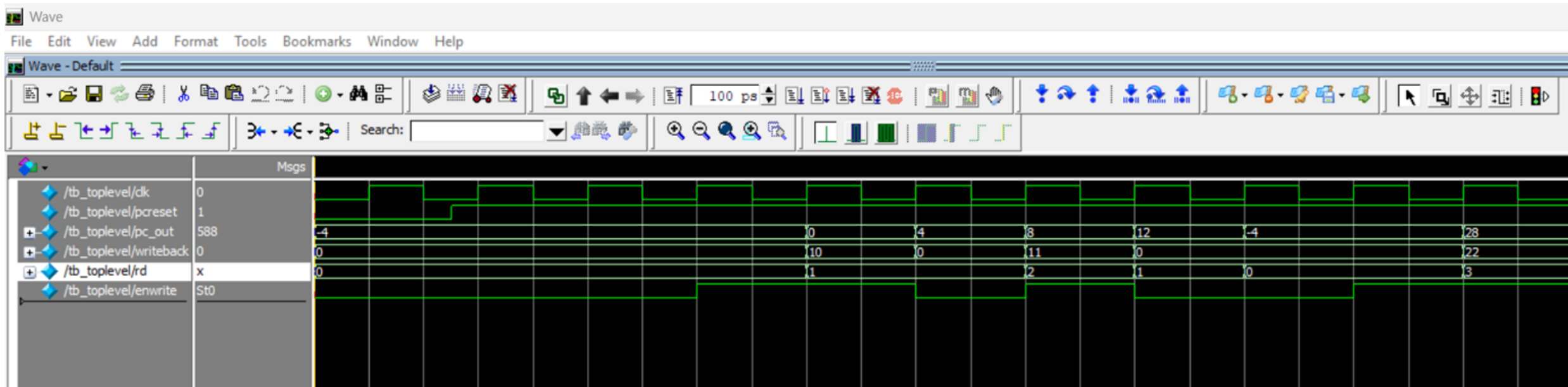
### 5.2 Test các lệnh I và Store Load

```
testcase5.asm
1  # 1. Khởi tạo
2  addi $1, $0, 25      # $1 = 25
3  andi $2, $1, 15      # EXPECTED: $2 = 9   (25 & 15 = 9)
4  ori  $3, $2, 16      # EXPECTED: $3 = 25   (9 | 16 = 25)
5  # 2. Ghi vào bộ nhớ (sw)
6  sw   $3, 0($0)       # Lưu số 25 ($3) vào địa chỉ RAM[0]
7  # 3. Đọc từ bộ nhớ (lw)
8  lw   $4, 0($0)       # Đọc RAM[0] vào $4 (Data = 25)
9  add  $5, $4, $2       # EXPECTED: $5 = 34 -> Mạch sẽ stall 1 nhịp
10 # 4. Test địa chỉ RAM khác
11 li   $6, -10         # EXPECTED: $6 = -10
12 sw   $6, 4($0)       # Lưu số -10 ($6) vào địa chỉ RAM[4]
13 lw   $7, 4($0)       # EXPECTED: $7 = -10
```



## 5.3 Test các lệnh rẽ nhánh

```
testcase4.asm
1  # Nhập dữ liệu
2  addi $1, $0, 10      # $1 = 10
3  # 1. BNE KHÔNG NHẢY (So sánh 10 với 10)
4  bne $1, $1, error    # ko nhảy (vì bằng nhau)
5  addi $2, $0, 11      # EXPECTED: $2 = 11
6  # 2. BEQ NHẢY (So sánh 10 với 10)
7  beq $1, $1, pass     # nhảy (vì bằng nhau)
8  addi $4, $0, 999      # EXPECTED: ko chạy
9  addi $5, $0, 999      # EXPECTED: ko chạy
10 error:
11 addi $4, $0, 999      # EXPECTED: ko chạy
12 pass:
13 addi $3, $0, 22       # EXPECTED: $3 = 22
```

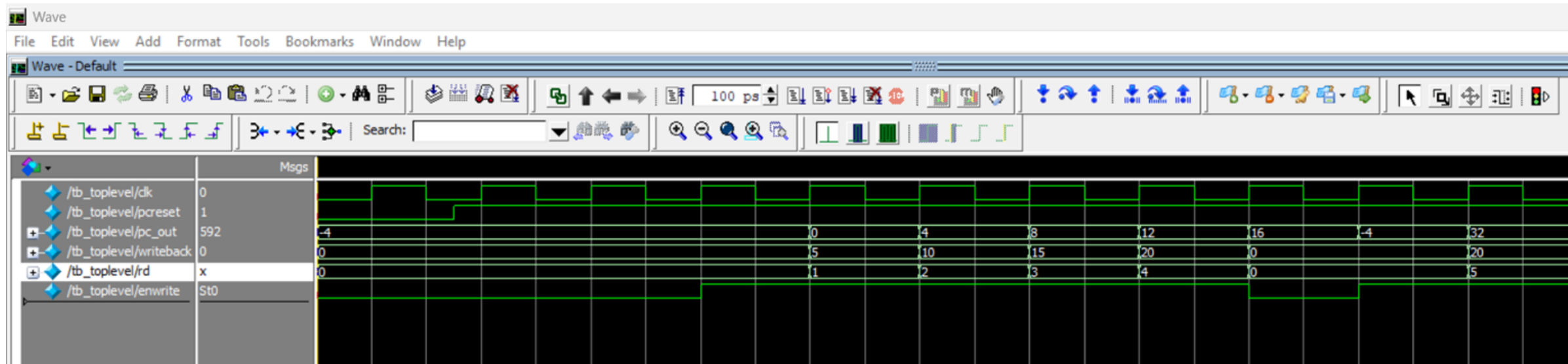




## 5. Kiểm tra kết quả mô phỏng

### 5.4 Test lệnh nhảy và Data Hazard

```
testcase6.asm
1  # 1. TEST DATA HAZARD
2  addi $1, $0, 5      # $1 = 5
3  add  $2, $1, $1      # EXPECTED: $2 = 10
4  add  $3, $2, $1      # EXPECTED: $3 = 15
5  add  $4, $3, $1      # EXPECTED: $4 = 20
6  # 2. TEST LỆNH JUMP (j)
7  j    jump_pass      # Nhảy qua 3 lệnh dưới
8  addi $4, $0, 999     # EXPECTED: ko chạy
9  addi $4, $0, 999     # EXPECTED: ko chạy
10 addi $4, $0, 999     # EXPECTED: ko chạy
11 jump_pass:
12 # Nếu nhảy đúng thì sẽ chạy dòng này
13 addi $5, $3, 5       # EXPECTED: $5 = 20 (15 + 5)
```





## 6. Đánh giá phần cứng và hiệu năng

### 6.1 Tài nguyên

|                                    |                                                 |
|------------------------------------|-------------------------------------------------|
| Quartus II 64-Bit Version          | 13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition |
| Revision Name                      | toplevel                                        |
| Top-level Entity Name              | toplevel                                        |
| Family                             | Cyclone II                                      |
| Device                             | EP2C35F672C6                                    |
| Timing Models                      | Final                                           |
| Total logic elements               | 2,967 / 33,216 ( 9 % )                          |
| Total combinational functions      | 2,464 / 33,216 ( 7 % )                          |
| Dedicated logic registers          | 1,493 / 33,216 ( 4 % )                          |
| Total registers                    | 1493                                            |
| Total pins                         | 72 / 475 ( 15 % )                               |
| Total virtual pins                 | 0                                               |
| Total memory bits                  | 8,192 / 483,840 ( 2 % )                         |
| Embedded Multiplier 9-bit elements | 0 / 70 ( 0 % )                                  |
| Total PLLs                         | 0 / 4 ( 0 % )                                   |



## 6. Đánh giá phần cứng và hiệu năng

### 6.2 Tần số

| Slow Model Fmax Summary |           |                 |            |      |
|-------------------------|-----------|-----------------|------------|------|
|                         | Fmax      | Restricted Fmax | Clock Name | Note |
| 1                       | 68.59 MHz | 68.59 MHz       | clk        |      |

$$T_{min} = \frac{1}{68.59 * 10^6 Hz} \approx 14.58 \text{ ns}$$



## 6. Đánh giá phần cứng và hiệu năng

### 6.3 Mô phỏng thời gian thực

```
testcase5.asm
1  # 1. Khởi tạo
2  addi $1, $0, 25      # $1 = 25
3  andi $2, $1, 15      # EXPECTED: $2 = 9   (25 & 15 = 9)
4  ori  $3, $2, 16      # EXPECTED: $3 = 25   (9 | 16 = 25)
5  # 2. Ghi vào bộ nhớ (sw)
6  sw   $3, 0($0)       # Lưu số 25 ($3) vào địa chỉ RAM[0]
7  # 3. Đọc từ bộ nhớ (lw)
8  lw   $4, 0($0)       # Đọc RAM[0] vào $4 (Data = 25)
9  add  $5, $4, $2       # EXPECTED: $5 = 34 -> Mạch sẽ stall 1 nhịp
10 # 4. Test địa chỉ RAM khác
11 li   $6, -10         # EXPECTED: $6 = -10
12 sw   $6, 4($0)       # Lưu số -10 ($6) vào địa chỉ RAM[4]
13 lw   $7, 4($0)       # EXPECTED: $7 = -10
```

